

Audio Processing Fundamentals

From sound waves to digital signals — the foundation of every audio AI system

9 min

Duration

B2 – Understand

Bloom's

L2 – Foundational

Level

Module 4.1

Prerequisite

Session Overview

What you will learn and how this fits in the curriculum

What You Will Learn

- What a sound wave is and how it becomes digital data
- Sample rate, bit depth, channels — the three digital audio parameters
- The Nyquist-Shannon theorem and why it matters
- Framing and windowing: preparing audio for spectral analysis
- The complete 6-step pre-processing pipeline used in production
- Common pitfalls engineers encounter in audio AI pipelines

Concept at a Glance

- Duration: 9 minutes
- Bloom's Level: B2 – Understand
- Difficulty: L2 – Foundational
- Module: 4.2 Audio and Speech Models
- Prerequisite: Module 4.1 (Multimodal AI Overview)
- Next Concept: Spectrogram and Audio Features

Learning Objectives

By the end of this session you will be able to:

9 min

Duration

B2 – Understand

Bloom's

L2 – Foundational

Level

Module 4.1

Prerequisite

1 Explain why raw audio waveforms cannot be fed directly to neural networks

2 Define sample rate, bit depth, and channel count and state their standard values for speech AI

3 Apply the Nyquist-Shannon theorem to determine the maximum capturable frequency

4 Describe framing and windowing and explain why 75% frame overlap is standard

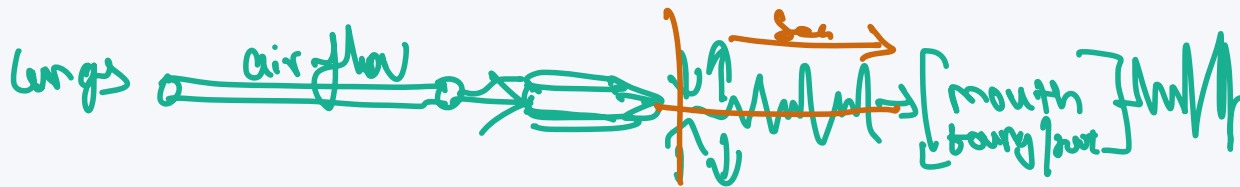
5 Trace the complete 6-step digital audio pre-processing pipeline used in production systems

SECTION 1 — THE PHYSICAL TO DIGITAL CHAIN

Sound Waves to Numbers

How physical pressure waves become the float32 arrays that neural networks consume

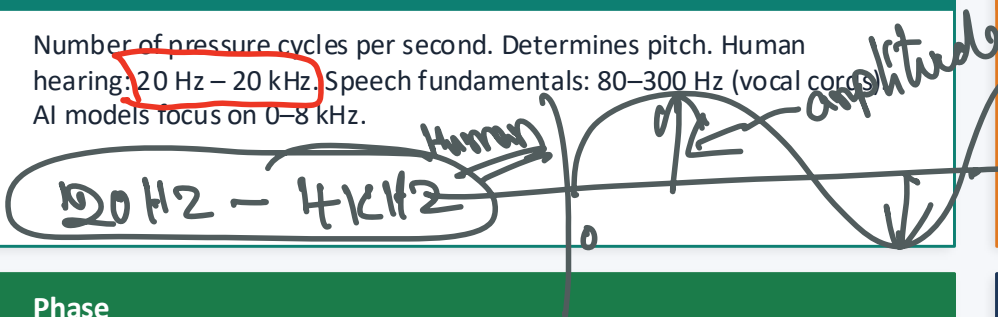
What Is a Sound Wave?



Pressure variations in air — characterised by frequency, amplitude, and phase

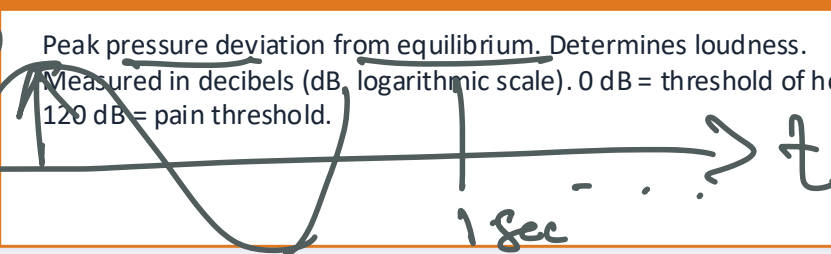
Frequency (Hz)

Number of pressure cycles per second. Determines pitch. Human hearing: 20 Hz – 20 kHz. Speech fundamentals: 80–300 Hz (vocal cords). AI models focus on 0–8 kHz.



Amplitude

Peak pressure deviation from equilibrium. Determines loudness. Measured in decibels (dB, logarithmic scale). 0 dB = threshold of hearing, 120 dB = pain threshold.



Phase

Offset of the wave relative to a reference. Critical for audio reconstruction (vocoder problem). Typically discarded during spectral analysis — a conscious trade-off.

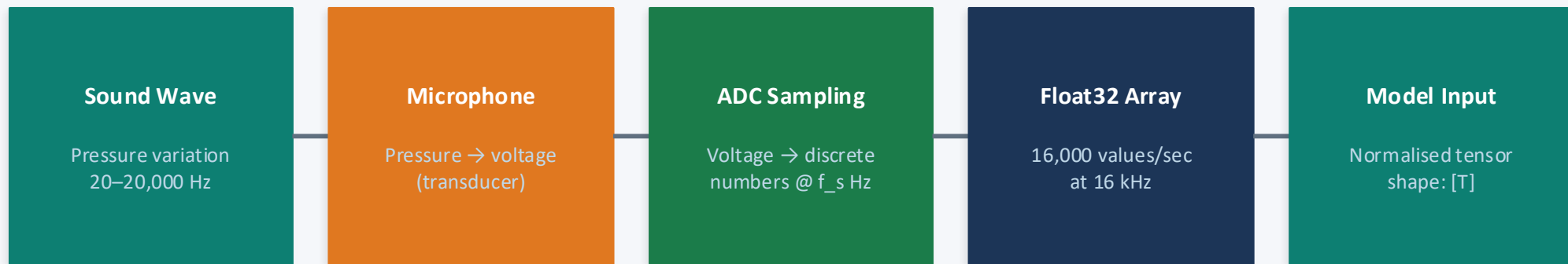
Waveform

The time-domain representation: amplitude vs time. A raw .wav file at 16 kHz stores 16,000 amplitude samples per second as float32 values in [-1.0, +1.0].

Key insight: A microphone is a transducer — it converts acoustic pressure into voltage. An ADC (Analog-to-Digital Converter) then samples that voltage at a fixed rate to produce the numeric array a model receives.

The Physical-to-Digital Chain

Every audio AI pipeline begins here — understanding this chain prevents the most common production bugs

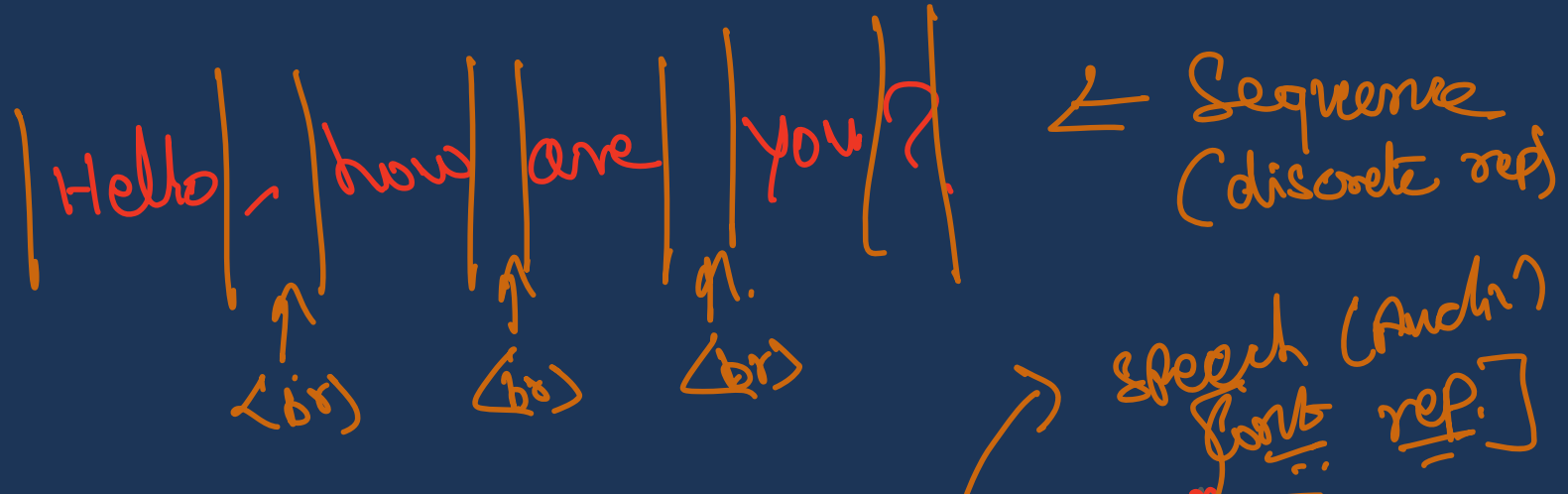


Engineer rule: `librosa.load(path, sr=16000, mono=True)` handles resampling and channel mixing automatically. Always specify `sr` explicitly — never assume the file matches your model.

SECTION 2 — THE THREE DIGITAL AUDIO PARAMETERS

Sample Rate, Bit Depth, and Channels

The three numbers that completely describe any digital audio signal



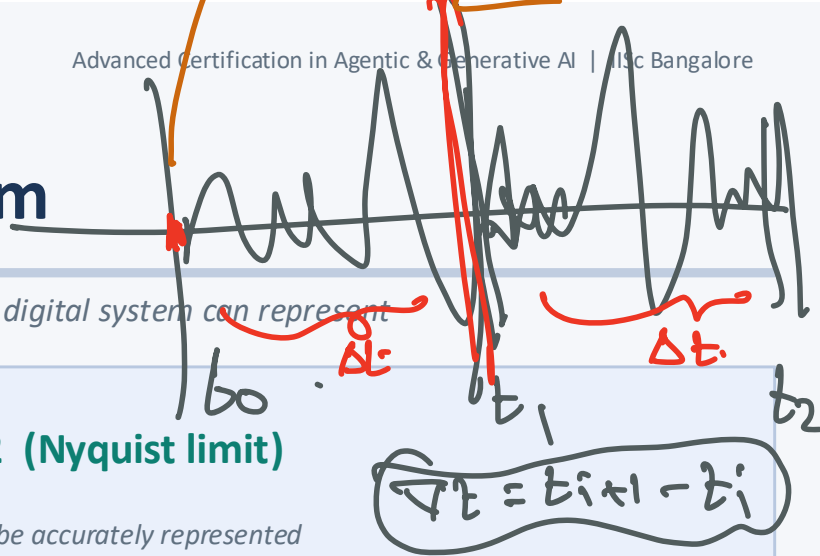
The Nyquist-Shannon Sampling Theorem

The mathematical foundation of digital audio — determines the maximum frequency a digital system can represent

$$f_s \geq 2 \cdot f_{\max} \Rightarrow f_{\max} = f_s / 2 \text{ (Nyquist limit)}$$

f_s = sample rate (Hz) | $f_{\max}$ = highest frequency that can be accurately represented

$$\Delta t = \frac{1}{2f_{\max}}$$



Sample Rate	Nyquist Limit	Primary Use Case
8,000 Hz	4,000 Hz	Telephone — captures speech vowels but loses sibilants /s/ /sh/
16,000 Hz	8,000 Hz	Speech AI standard — Whisper, wav2vec, HuBERT, Conformer
22,050 Hz	11,025 Hz	TTS output — FastSpeech 2, HiFi-GAN vocoder default
44,100 Hz	22,050 Hz	CD audio / music — covers full human hearing range
48,000 Hz	24,000 Hz	Professional recording, broadcast, video audio tracks

ASR

TTS

Why 16 kHz? Speech fundamentals are 80–300 Hz, formants up to ~5 kHz, sibilants up to ~8 kHz. 16 kHz captures everything humans use for intelligibility. Higher rates add computation with no benefit for ASR.

Bit Depth and Channels

Two more parameters that define audio quality — both have standard values for AI systems

Parameter	Options	AI Standard	Why
Bit Depth	8-bit (256 levels), 16-bit (65,536 levels) 32-bit float, 24-bit studio	32-bit float (internally)	Models compute in float32. Raw audio loaded as 16-bit PCM and cast to float32 on load.
Channels	Mono (1) Stereo (2) 5.1 surround (6) Multi-mic (N)	Mono	ASR models expect mono. Stereo is mixed to mono: $\text{mono} = (L + R) / 2$. Spatial info is lost.
Duration	Any length	30-sec chunks (Whisper)	Long sequences are split to manage memory. Chunk with 50% overlap to avoid cutting words at boundaries.
File Format	.wav .flac .mp3 .ogg .m4a	WAV/FLAC (lossless)	MP3 compression introduces codec artefacts visible as checkerboard patterns in spectrograms.

Production checklist: On loading any audio file: verify sr == 16000, channels == 1, dtype == float32, $\max(\text{abs}(\text{audio})) \leq 1.0$. Log and reject files that fail any check.

SECTION 3 — FRAMING AND WINDOWING

Preparing Audio for Spectral Analysis

Why we must cut long signals into short overlapping frames before applying the Fourier Transform

Why Framing Is Necessary

A 5-second audio clip at 16 kHz = 80,000 samples — too long and non-stationary for direct FFT

"30 sec"

Non-stationarity Problem

Speech frequencies change continuously. The /s/ in 'speech' is at 6–8 kHz; the /ee/ vowel is at 200–2000 Hz. A single FFT over 5 seconds blurs all these together — time information is lost.

Computational Size Problem

FFT complexity is $O(N \log N)$. For $N=80,000$ samples, this is expensive and produces an unwieldy 40,000-point frequency spectrum. We need a compact, manageable representation.



The Solution: Short-Time Framing

Divide the signal into short overlapping frames of ~25 ms each (400 samples at 16 kHz). Within each 25 ms window, speech is approximately stationary — frequencies don't change significantly in 1/40th of a second. Apply FFT to each frame independently, then stack the results column-by-column to form the spectrogram matrix.

Standard parameters: Frame length = 25 ms (400 samples at 16 kHz). Hop length = 10 ms (160 samples). Overlap = 60%. These values were established empirically as optimal for speech — used by Whisper, Kaldi, librosa.

Window Functions: Eliminating Spectral Leakage

Abrupt frame edges cause 'spectral leakage' — energy from one frequency appears to bleed into others

Spectral Leakage: When a frame is cut with abrupt edges, the FFT 'sees' discontinuities and distributes energy from a single pure tone across many frequency bins — corrupting the spectrum. Window functions taper the frame to zero at both edges, eliminating this artefact.

Rectangular

Formula: $w(n) = 1$ for all n

✓ Maximum frequency resolution. Simple.

✗ Severe spectral leakage. Never used in audio AI.

Hann

Formula: $w(n) = 0.5 \cdot (1 - \cos(2\pi n/N))$

✓ Excellent leakage suppression. Smooth taper to zero. Used by Whisper, librosa (default).

✗ Slightly reduced frequency resolution at edges.

Hamming

Formula: $w(n) = 0.54 - 0.46 \cdot \cos(2\pi n/N)$

✓ Similar to Hann but never reaches zero. Better amplitude accuracy. Kaldi default.

✗ Slightly more leakage than Hann at bin edges.

Recommendation: Always use the Hann window unless you have a specific reason not to. `librosa.stft()` uses Hann by default. Whisper uses Hann. Any modern tutorial that says 'Hamming' is citing older Kaldi-era practice.

SECTION 4 — THE PRODUCTION PIPELINE

Complete Audio Pre-Processing Pipeline

Six steps that every production audio AI system executes before passing audio to the model

The 6-Step Audio Pre-Processing Pipeline

A missed or wrong step causes silent errors downstream — most audio AI bugs originate here

1

Load & Resample

Load any format (mp3, wav, flac, m4a) and resample to 16,000 Hz mono.
Code: `y, sr = librosa.load(path, sr=16000, mono=True)`

2

Validate

Assert `sr==16000, y.dtype==float32, y.ndim==1, abs(y).max()<=1.0`
Reject or flag files that fail. Log the failure with filename.

3

Normalise Amplitude

Peak: $y = y / \max(\text{abs}(y)+1e-8)$. RMS: $y = y * (\text{target_rms} / \text{rms}(y))$
Prevents gradient issues from volume variation across recordings.

4

Pre-emphasis

`y_pre = np.append(y[0], y[1:] - 0.97*y[:-1])`
Boosts high frequencies before FFT to compensate for natural high-freq roll-off in speech.

5

Frame & Window

`frames = librosa.util.frame(y, frame_length=400, hop_length=160)`
`frames = frames * np.hanning(400)[:,:].newaxis`
25 ms frames, 10 ms hop, 75% overlap.

6

VAD (Optional)

Voice Activity Detection: Silero VAD or WebRTC VAD
Remove silence frames. Reduces compute by 30–50% and prevents blank-token inflation in ASR.

Debug tip: After step 3 (normalise), plot a histogram of `y` values. Should be roughly Gaussian centred at 0, bounded within ± 1.0 . Any values outside ± 1.0 mean normalisation failed.

SECTION 5 — HOW THIS IS USED IN PRODUCTION

Audio Processing in Real AI Systems

Where these fundamentals appear in production speech and audio AI systems

Production Systems Using These Fundamentals

Every major audio AI system applies the same chain — only the subsequent representation differs

Systems and Their Audio Configuration

Whisper (OpenAI) 16 kHz mono, 30-sec chunks, Hann window 400/160, 80-bin log-Mel. Resample and pad to exactly 30 sec before inference.

wav2vec 2.0 (Meta) 16 kHz mono raw waveform fed directly to CNN frontend. No STFT — the model learns its own frontend filters. Still needs normalisation.

Conformer ASR (Google) 16 kHz mono, 25 ms/10 ms framing, 80-bin Mel. On-device: quantised to int8. SpecAugment applied to Mel during training.

Tacotron 2 / FastSpeech 22,050 Hz for TTS output quality. 80-bin Mel spectrogram as intermediate target. HiFi-GAN vocoder converts Mel back to waveform.

Common Production Pitfalls

Wrong sample rate Model receives audio at 44.1 kHz when it expects 16 kHz — frequencies are halved. Produces word error rates of 70%+. Always check.

Stereo not mono-ised Model receives [2, T] instead of [T] tensor — shape mismatch or averaged incorrectly. Mix before normalisation, not after.

MP3 codec artefacts Checkerboard patterns in spectrogram at regular intervals. Use WAV/FLAC for training data. Detect with pdfimages on spectrograms.

Clipped audio Values > 1.0 or < -1.0 after loading. Usually from incorrect gain staging. Cast to float32 and clip before normalising.

Debugging rule: When ASR WER is unexpectedly high on a specific file, the first check is always: `print(sr, y.shape, y.dtype, y.min(), y.max())`. 80% of audio bugs are caught here.

Common Misconceptions

Critical clarifications on concepts that are frequently confused at this stage

MYTH: 'Higher sample rate always means better AI performance'

CORRECTION: For speech AI, 16 kHz is optimal. Going to 44.1 kHz quadruples the data size and FFT cost with no additional useful speech information. The benefit is only for music or audio with content above 8 kHz.

MYTH: 'Normalisation is optional — my model can handle any amplitude'

CORRECTION: False. Unnormalised audio causes gradient explosions in training and large variation in inference confidence. Always normalise. The only question is whether to use peak or RMS normalisation.

MYTH: 'Pre-emphasis is a legacy step from HMM systems — modern DL doesn't need it'

CORRECTION: Partially true. Deep learning models (Conformer, wav2vec) can learn to compensate for high-frequency roll-off implicitly. However, pre-emphasis still improves convergence speed in many settings. Whisper does NOT apply it — librosa's default does.

Key Takeaways

The five things you must remember from this concept

Sample rate determines frequency range 16 kHz is the speech AI standard — captures 0–8 kHz, covering all intelligible speech content (Nyquist limit = $f_s / 2$).

Bit depth determines dynamic range 32-bit float internally; 16-bit PCM on disk. Always cast to float32 on load and verify range is $[-1.0, +1.0]$.

Framing solves non-stationarity 25 ms frames, 10 ms hop (75% overlap). Short enough for speech to be stationary within each frame.

Hann window eliminates spectral leakage Tapering frame edges to zero prevents energy from one frequency bleeding into adjacent bins in the FFT output.

The 6-step pipeline is universal Load → Validate → Normalise → Pre-emphasis → Frame+Window → (STFT). Every audio AI system from Whisper to wav2vec applies this chain.

Note: Next concept: AI-MM-IN-TH-000007 — Spectrogram and Audio Features (STFT, Mel filterbank, MFCCs)

Prerequisites and What This Unlocks

Understanding the knowledge graph position of this concept

You Should Already Know

- Basic understanding of what AI/ML is (Module 1)
- What a neural network input tensor is
- Python and NumPy basics (array operations)
- High-level awareness of speech applications

This Concept Unlocks

- AI-MM-IN-TH-000007: Spectrogram and Audio Features
- AI-MM-IN-TH-000008: ASR Models (Conformer, Whisper)
- AI-MM-IN-TH-000009: TTS Systems (Tacotron, FastSpeech 2)
- AI-MM-IN-CD-000002: Building ASR/TTS Applications
- AI-MM-IN-TH-000011: Audio-Visual Learning

Position in curriculum: This is the entry-level audio concept. It deliberately stays at signal level — no machine learning yet. The goal is to make every subsequent audio AI concept interpretable.

AI-MM-IN-TH-000006 Complete

Audio Processing Fundamentals

Advanced Certification in Agentic & Generative AI · IISc Bangalore